

Knowledge-and-Language 2025/2026

Project Report

A Bilingual Recipe Assistant Combining a Knowledge Graph, Lightweight NLP, and LLMs

Miguel Castela, uc2022212972
DEI, Universidade de Coimbra
uc2022212972@student.uc.pt

Miguel Martins, uc2022213951
DEI, Universidade de Coimbra
uc2022213951@student.uc.pt

I. ABSTRACT

We present a bilingual recipe assistant that combines a curated recipe Knowledge Graph (KG), intent classification, entity extraction and SPARQL query retrieval. This ends in an LLM generating the summary of the results in the user's language. In this report, we describe the data curation, approach, implementation, and experiments, highlighting strengths, limitations, and future directions.

II. INTRODUCTION

Motivation.

Recipe search is often unstructured and specific. Users can query for ingredients, dietary tags, or time constraints. Most systems either rely on pure keyword search or naive large language model answers. We aim to combine structured knowledge graph retrieval with natural language generation, ensuring factual retrieval and usability.

Problem Description. Given a user query in Portuguese or English, the system must:

- Detect intent;
- Extract and link entities to the KG via fuzzy matching;
- Retrieve candidate results via SPARQL;
- Perform a RAG similarity search over recipe descriptions;
- Generate summaries based on all the information, the user's query and the recent history;
- Present results through an interactive interface.

Goal and Approach Our approach is building the following pipeline:

- A curated recipe knowledge graph in RDF/Turtle;
- An intent classifier trained with BERT;
- An NLP pipeline for entity extraction and fuzzy linking;
- A FAISS embedding index for recipe descriptions;
- LLM-based summarization;
- A React/Vite-based frontend.

III. RELATED WORK

Research on integrating symbolic knowledge representations with neural language models has expanded significantly in recent years, particularly in the context of improving factual accuracy and domain-specific reasoning. A central issue identified by Agrawal et al. [1] is that Large Language Models (LLMs) often struggle with domain grounding due to limitations in context comprehension, factual knowledge, and reliable recall. This has motivated work on hybrid systems that augment neural architectures with structured knowledge resources such as RDF/OWL Knowledge Graphs (KGs) and external retrieval modules. For this effect RDF encodes information as subject–predicate–object triples, enabling flexible graph-based modelling [2]. RDFS adds schema semantics such as class hierarchies and domain/range constraints, while OWL introduces richer logical constructs including cardinality restrictions, equivalence, and disjointness [2].

In these hybrid systems, SPARQL querying allows precise retrieval of RDF data [2], while either NLP rule based systems or more complex tools such as spaCy [3] and BERT [4] enable entity recognition, intent classification, and mapping between natural language and the KG entities.

A major line of research addressing LLM factuality is Retrieval-Augmented Generation (RAG). Lewis et al. [5] introduced RAG as a hybrid architecture combining a parametric generator (BART [6]) with non-parametric memory built from Dense Passage Retrieval (DPR) [7]. The RAG-Sequence and RAG-Token variants further refine retrieval–generation interactions, with the latter enabling token-level grounding from different retrieved documents [5]. This framework has become a widely adopted strategy for mitigating hallucinations in neural generation.

After researching various approaches to integrating knowledge graphs with LLMs for domain-specific question answering,

we identified RecipeRAG [8] as a particularly relevant prior work for our project. RecipeRAG addresses the challenge of answering complex recipe queries by combining structured knowledge graph embeddings with generative modelling. KGEs such as TransE [9], RotatE [10], and QuatE [11] encode entities and relations into continuous vector spaces, enabling similarity-based retrieval that respects graph structure. RecipeRAG demonstrates that KGE-driven retrieval outperforms purely text-based similarity when handling multi-criteria recipe search, and further integrates retrieved KG evidence into a generative model to produce personalized recipes. This work highlights the potential of combining symbolic knowledge, embedding-based retrieval, and generative modelling in domain-specific applications such as recipe recommendation.

Given this context, the project proposed in this report builds on these research directions by integrating (i) an RDFS-based KG, (ii) an NLP pipeline using BERT [4] and spaCy [3] for intent and entity extraction, and (iii) a retrieval step inspired by RAG [5], enriched with structured recipe data. Similar to RecipeRAG [8]. Instead of relying solely on knowledge graph embeddings for retrieval, our approach combines precise SPARQL querying with an embedding similarity search over recipe descriptions. This hybrid architecture aims to leverage the strengths of structured knowledge representation, lightweight NLP techniques, and LLM generation to provide more accurate, context-aware answers to bilingual recipe queries based on both structured and unstructured data.

IV. DATA SELECTION

Firstly we searched for a dataset with portuguese recipes, but the ones we found were poorly structured. ([12]) is a portuguese cuisine dataset that unfortunately did not meet our requirements for structure and completeness.

Thus, we opted to use a portion of the Food.com [13] dataset that, in whole, contains over 200,000 recipes,

The dataset was already curated because its based on the website data that contain only recipes with the following information:

TABLE I: Fields present in each recipe entry.

Field	Description
name	Recipe name
id	Unique recipe identifier
minutes	Total preparation/cooking time
contributor_id	ID of the user who submitted the recipe
submitted	Submission date
tags	List of categorical tags
nutrition	Nutritional information vector
n_steps	Number of preparation steps
steps	Ordered list of preparation steps
description	Short user's recipe description
ingredients	List of ingredients
n_ingredients	Number of ingredients

However, as stated in the project proposal, we aim to have a bilingual dataset containing recipes in either portuguese or english languages. This is necessary to then be able to make

accurate similarity retrievals and SPARQL queries based on user queries in either of these two languages.

This filtering was achieved using two methods

- Firstly, some portuguese recipes were automatically matched using a set of specific keywords defined by hand (for example: "bacalhau", "lisbon", "azores") using regular expressions. In this stage, a pre-trained langid model was also used (ENCOMENDA REAL) to classify portuguese recipes by their full sentences, with a probability score of above 0.8. In this stage
- A lot of false negatives were still present from the first step. A pre-trained fasttext LLM (lid.176.bin) (ENCOMENDA REAL) was used to predict the language of each token from the data of each recipe. To be labeled as portuguese a token would have to be considered as 'PT' with a confidence ≥ 0.7 and the difference to the second detected language had to be ≥ 0.15 .

Both datasets were merged, however there was still a relatively high rate of false positives, a total of around 1435. This was partially due to a lot of spanish recipes being classified as portuguese on account of the similarity of the two languages. This problem was solved with a pass of the merged dataset through an LLM (chatGPT) (ENCOMENDA REAL) to classify the recipes as portuguese or not portuguese.

After removing the false positives 1015 portuguese recipes remained. After this around 9000 random recipes were added to the dataset. To keep portuguese and english recipes (VER O METODO DE TIRAR RECEITAS QUE N SEJAM INGLES)

V. APPROACH

A. Knowledge Graph Construction

After the final dataset of around 10,000 recipes was obtained, the knowledge graph was generated in RDFS format. For the graph generation only the following fields were used as RDFS classes: name, id, minutes, tags, nutrition, n_steps, steps, ingredients, n_ingredients. This was the decided structured considered most relevant to build the proposed chatbot.

B. Intent Training

The first step to analyse the user's query is to detect its intent. A list of five possible intents was defined, and a dataset of around 1,100 training examples was generated using seed samples and an LLM. This dataset contained both portuguese and english examples. The following table shows some examples of user queries and their associated intents:

In the dataset building, it was necessary to ensure a clear separation between similar cases such as "recipes that take 30 minutes" (list_by_time) and "how long does arroz de pato take to cook?" (get_prep_time).

C. Entity Extraction

The next task is to extract relevant entities from the user query in order to have a base to build the SPARQL queries. This is done using a similarity search with fuzzy matching

TABLE II: Examples of user queries and their associated intents.

User Query	Intent
Quanto tempo para cozer peito de frango	get_prep_time
Find low-carb meals	list_by_tag
What do I need for paella	retrieve_ingredients
Como fazer francesinha em casa	find_recipe
Pratos para cozinhar em 45 minutos	list_by_time

against the knowledge graph entities. The entites are extracted according to the predicted intent that has the most confidence. The entities that are considered and thus passed to the pipeline are ordered according to their similarity score, if above a certain threshold. The following table shows the mapping of intents to the entities extracted by the pipeline:

TABLE III: Mapping of intents to the entities extracted by the pipeline.

Intent	Extracted Entity / Slot
get_prep_time	recipe_name
list_by_tag	tag
retrieve_ingredients	recipe_name
find_recipe	recipe_name
list_by_ingredient	ingredient
list_by_time	minutes

D. SPARQL Queries

The valid entities extracted from the user query are then used to build SPARQL queries to retrieve the desired information from the knowledge graph. Each intent has its own SPARQL query template that is filled with the extracted entities. The following graphs shows the mapping of intents to their corresponding SPARQL query functions:

(IMAGENS DOS QUERIES)

E. Retrieval Augmented Generation

As the descriptions of the recipes in the knowledge graph are more textual and less structured,a RAG (ENCOMENDA REAL) (retrieval augmented generation) system was implemented to be able to retrieve the most relevant descriptions based on the user query.

F. LLM Contextual Response

Finally, the retrieved information from the SPARQL queries and the RAG system is used to generate a summary response to the user query. This is done using an LLM that takes as input the user query, the retrieved recipes, and their descriptions. The LLM then generates a coherent and informative response that answers the user’s query based on the retrieved information and past conversation history.

VI. IMPLEMENTATION

This section will explain in detail how each of the components described in the approach section were implemented.

A. Knowledge Graph Construction

After pre-processing the dataset the next step is to generate a knowledge graph in RDFS format. The script built for this task uses the RDFlib library (ENCOMENDA REAL) and converts the CSV into an RDF/Turtle knowledge graph by creating cleaned URIs and structured entities for recipes, ingredients, tags, steps, and nutrition, then serializes the result to a .ttl file. This is an example of how a recipe is represented in the knowledge graph:

```
@prefix ex: <...> .
@prefix rdf: <...> .
@prefix rdfs: <...> .
@prefix xsd: <...> .

ex:Ingredient a rdfs:Class .

ex:Nutrition a rdfs:Class .

ex:Recipe a rdfs:Class .

ex:Step a rdfs:Class .

ex:Tag a rdfs:Class .

ex:Recipe_100002 a ex:Recipe ;
    rdfs:label "chips ahoj chocolate pie" ;
    ex:hasIngredient ex:Ingredient_butter,
        ex:Ingredient_chocolate_
        chip_cookies,
    ex:hasNutrition ex:Recipe_
    100002_Nutrition ;
    ex:hasStep ex:Recipe_100002
    _StepsSeq ;
    ex:hasTag ex:Tag_60_minutes
    _or_less,
        ex:Tag_course,
    ex:id 100002 ;
    ex:minutes 40 ;
    ex:n_ingredients 5 ;
    ex:n_steps 10 ;
    ex:origin "original randomized" .

ex:Recipe_100002_Nutrition a ex:Nutrition ;
    ex:calories 3.581e+02 ;
    ex:protein 7e+00 ;
    ex:saturatedFat 6.3e+01 ;
    ex:sodium 1.5e+01 ;
    ex:sugar 4.2e+01 ;
    ex:totalFat 3.5e+01 ;
    ex:unknown 1.1e+01 .

ex:Recipe_100002_Step_1 a ex:Step ;
    rdfs:comment "crush 16 cookies in a
    ↪ sealable plastic bag until fine crumbs
    ↪ form" .

ex:Recipe_100002_Step_2 a ex:Step ;
    rdfs:comment "mix 1 1 / 2 cups of the
    ↪ crumbs with melted butter" .

ex:Recipe_100002_StepsSeq a rdf:Seq ;
    rdf:_1 ex:Recipe_100002_Step_1 ;
    rdf:_2 ex:Recipe_100002_Step_2 ;
```

```

ex:Ingredient_chocolate_chip_cookies a
  ↪ ex:Ingredient ;
  rdfs:label "chocolate chip cookies" .

ex:Tag_60_minutes_or_less a ex:Tag ;
  rdfs:label "60-minutes-or-less" .

ex:Tag_course a ex:Tag ;
  rdfs:label "course" .

```

As seen from the example above, each recipe is represented as an instance of the `Recipe` class, with properties linking to the necessary components. Some of these components have their own classes, as shown below:

TABLE IV: Recipe class attributes & RDFS classes.

Recipe Attributes	RDFS Classes
recipe	Recipe
ingredient	Ingredient
tag	Tag
step	Step
nutrition	Nutrition
id	
minutes	
n_ingredients	
n_steps	

This structured representation allows for the reuse of entities like ingredients and tags across multiple recipes without duplication. The steps were transformed into a RDF sequence (`rdf:Seq`) as shown in the example above. To maintain their order, each step is linked to the recipe using the `ex:hasStep` property. Nutrition attributes (calories, protein, saturated fat etc) are encapsulated in a separate `Nutrition` instance linked to the recipe via the `ex:hasNutrition` property. Tags are linked using the `ex:hasTag` property, allowing for categorization and filtering of recipes based on various user tags. This allows for efficient querying and easier retrieval of the relevant information for the predicted intent.

A small subset with two recipes and their full attribute was put into a ttl tag for observation using the `pyvis` library after parsing, (the snippet above is taken from this example) This is shown in the figure below:

(IMAGE DO GRAFO)

B. Intent Training

The first step to analyse the user’s query is to detect its intent. In order to do so an intent classifier was trained using a multilingual BERT model (ENCOMENDA REAL) and the LLM generated dataset mentioned in the section above.

Using this dataset, we trained a multilingual BERT (ENCOMENDA REAL) intent classifier: it loads the labeled CSV, tokenizes the text with `bert-base-multilingual-cased`, builds label

mappings, fine-tunes a sequence-classification model, evaluates it on an 80/20 split, and finally saves the trained model, tokenizer, and label map for later use in the pipeline.

C. Entity Extraction

Alongside the classifier, we developed an entity extractor that uses `spaCy` (ENCOMENDA REAL) for NER (name entity recognition) and `RapidFuzz`(ENCOMENDA REAL) for fuzzy matching against the Knowledge Graph entities. As mentioned before, this allows for the extraction of relevant ingredients, tags, recipe names, and time constraints from the user query based on the predicted intent.

The `list_by_time` intent required special attention because user descriptions of preparation time are highly variable and often ambiguous. Unlike other intents that rely on explicit lexical markers, time expressions can appear in many different forms: numerical values (“30 minutos”), written numbers (“trinta minutos”), comparative forms (“menos de 20 minutos”, “under 10 minutes”), vague descriptions (“algo rápido”, “something quick”), and mixed-unit expressions such as “1h20” or “1 hora e meia”. In addition, user queries may omit units, combine formats, or express constraints indirectly, which makes it difficult to rely on a single pattern or keyword. The system therefore has to distinguish between exact-time queries (asking for recipes that take exactly a specific number of minutes) and maximum-time queries (asking for recipes that take at most a certain amount of time). It also must interpret comparative structures correctly and normalize written forms into numeric values. For these reasons, the `list_by_time` logic integrates layered numeric parsing, distinction between exact and maximum constraints, translation normalization, and fallback mechanisms so that the fuzzy tag search can be used when no explicit time is detected.

- Language detection and normalization:
 - Detects query language (PT/EN) using the `fasttext` model (ENCOMENDA REAL) after detection using Portuguese heuristics (accents, “ç” and common PT words);
 - Strips accents for robust matching while preserving original forms for display.
- `spaCy` pipeline:
 - Loads the (PT/EN) `spaCy` models;
 - Adds a flag for keywords related to time (min/minutos/hora/hours);
 - Extracts noun chunks, preserving order.
- Cooking-time parsing in the `list_by_time` intent (PT/EN, tolerant to typos):
 - Normalizes number words (e.g., “quarenta e cinco”, “forty-five”) into digits;
 - Uses regexes to detect minutes, hours, or combined “H+M” formats for the minutes component (e.g., “1h 30m” to “90”);
 - Detects range prefixes (e.g., “under”, “menos de”, “até”) to separate exact vs. `max_minutes`.

- Translation with protected culinary terms:
 - Uses Argos Translate offline (ENCOMENDA REAL) for PT↔EN,
 - A dual pass is used for objective results no matter the input language, since the KG is bilingual but has more EN labels;
 - Preserves protected PT terms (e.g., “açorda”, “brás”) by not translating them.
- Candidate tokenization (`content_tokens`):
 - Removes stopwords and domain noise; sorts tokens for canonical matching;
 - Aggregation of tokens from original chunks, full text, and its translation.
- Knowledge Graph indexing:
 - Loads the KG labels into rdflib (ENCOMENDA REAL);
 - Precomputes accent-stripped normalized labels for fast fuzzy search.
- Intent-aware fuzzy search and linking (`()`):
 - RapidFuzz (ENCOMENDA REAL) over normalized labels with thresholds: `ingredient=85`, `tag=90`, `recipe_name=70`.
 - Portuguese labels are preferred with an increment of 3 percentual points when the query appears to be PT (this way portuguese labels are favored).

After this the retrieved information is used to generate different SPARQL queries for each intent. These are explained in the following section:

D. SPARQL Queries

E. Pipeline

This file detects language, original and translated query, and detected slots before SPARQL execution. After, it runs the sequential pipeline and prints per-intent SPARQL outputs; returns a structured dict containing `intent`, `confidence`, `text`, `slots`, and `kg_results`.

- Sequential query execution (`handle_query`):
 - Determines the user intent and extracts slots with `extract_and_link`, then routes execution to the appropriate SPARQL query based on the intent.
 - `list_by_ingredient`:
 - * Collects all ingredient labels and runs `query_list_by_ingredient` for each.
 - `list_by_tag`:
 - * Uses all extracted tag labels, executes `query_list_by_tag`, and deduplicates recipes by URI.
 - `find_recipe`:
 - * For each candidate recipe name, calls `query_find_recipe` and prints a full recipe card.
 - `retrieve_ingredients`:

- * Calls `query_retrieve_ingredients` for each recipe name and prints the ingredient list.
- `get_prep_time`:
 - * For each recipe name, runs `query_get_prep_time` and prints the preparation time in minutes.
- `list_by_time`:
 - * Reads `cooking_time` (exact) or `max_minutes` (range) from the slots.
 - * Range queries use `query_by_max_minutes`; exact queries use `query_by_exact_minutes`; sets `result_basis`.
 - * If no results or no numbers are detected, performs a fuzzy tag fallback: injects time-related tags (15/30/60-minutes-or-less), queries `query_list_by_tag`, deduplicates, and sets the result to `fuzzy_tag_fallback`.

VII. RAG IMPLEMENTATION

As mentioned before, this module builds and serves a vector-based semantic search index over the more textual recipe descriptions. Before indexing, the code applies a quality-filtering step as short, boilerplate, or low-information descriptions are detected using heuristics such as length, alphabetic-character ratio, presence of boilerplate patterns (e.g., `prep time`, `yield`), and token count. Any description that fails these checks is skipped. Valid descriptions are then turned into “chunks”. In this version, each description is a single chunk which is stored in a list of records containing the fields `{"id", "name", "chunk_id", "text"}`. The build step encodes all chunks using a SentenceTransformer (ENCOMENDA REAL) mode. The resulting embeddings are L2-normalized and saved to `embeddings.npy`. A FAISS inner-product index is then created and written to `faiss.index`. Alongside it, a metadata file is generated so it can be used later for querying. This whole process only needs to be run once, and the resulting files are stored for later queries. When querying, the index and metadata are lazily loaded and cached. A query string is encoded using the same model and the FAISS index retrieves the most similar chunks. Extra candidates are fetched and deduplication is employed by their `chunk_id`. Results below a similarity threshold are filtered out. The returned list includes the recipe ID, recipe name, chunk ID, the original description text, and the similarity score.

VIII. DECISION OF LLM SUMMARIZATION MODULE AND IMPLEMENTATION

Initial tests with the local Ollama (ENCOMENDA REAL) model proved unsatisfactory. Despite the advantage of running locally without relying on an external service, the models demonstrated semantic inconsistencies with a more critical issue being the amount of time required for responses for each query. Consequently, the implementation was migrated

to the Groq API (ENCOMENDA REAL), which provided, with the use of significantly bigger models, improved response accuracy, coherence across turns, and overall stability in the time it took to process and generate the expected output. The Groq API is initialized lazily, ensuring that the module is loaded only when required. The file that implements this module does the following steps:

- conditional loading of environment variables;
- lazy loading of the semantic pipeline (ensuring it’s only initialized once and reused);
- integration with the RAG similarity engine;
- prompt construction with history and response policy;
- sending requests to Groq via `client.chat.completions`;
- logging conversation history in `context.txt`;
- maintaining a running session.

A. Frontend details

The frontend of the recipe chatbot was designed with several key user-focused features. Users can create a new chat, which clears the message history stored in the Groq prompt, ensuring that subsequent queries start with a fresh context. While the model is retrieving the context and generating the response, the interface allows users to view the previous message history for reference. The displayed text on the interface can also be dynamically translated. Additionally, all LLM responses are converted to Markdown for rendering, enabling proper formatting of tables and structured content. These frontend features are integrated with the backend described earlier, which is exposed via a FastAPI server (ENCOMENDA REAL). The API has endpoints for health checks, submitting chat queries, and clearing the conversation context. The frontend itself is implemented in React, with a dedicated `fonts` and `components` folder to manage the styling and reusable UI elements.

IX. PROBLEMS & SOLUTIONS

A. Deduplication of results

One challenge encountered during the development of the pipeline was the handling of duplicate recipe results arising from overlapping filtering criteria. The knowledge graph’s structure and tagging system make it possible for a single recipe to satisfy multiple filtering criteria at once—for example, a dish might be labeled both as “quick” and “under 30 minutes,” or appear in multiple result sets inferred through different SPARQL paths. Without explicit handling, such overlaps would cause duplicated recipe entries in the output, misleading users and complicating evaluation of recall and precision. To solve this issue, the pipeline applies a deterministic de-duplication pass (as mentioned above) after SPARQL results are collected. It iterates over the returned recipes, tracks seen URIs in a set, and retains only the first occurrence of each unique recipe. This lightweight but effective mechanism guarantees that results remain clean, non-redundant, and semantically meaningful.

B. Translation normalization

In the early versions, the pipeline relied on a plain large-language-model translation step to normalize user queries into english before intent classification and entity extraction. While this worked reasonably well, it produced non-deterministic translations as the same portuguese phrasing translated into english would not necessarily translate back into the same portuguese string, and small paraphrases in the input would sometimes yield semantically equivalent but structurally different english or portuguese renderings. This inconsistency became problematic because the the spaCy entity extractor depends on predictable lexical patterns. To address this normalization measures were introduced to make the translation stage behave more objectively.

C. Language and Number normalization

This began by removing stopwords that added noise and led the LLM to produce stylistic rather than literal translations. Secondly, instead of translating token by token, which caused the LLM to reinterpret fragments, the entire sentence was translated as a single unit. Afterwards, we fixed a systematic discrepancy in number handling as PT→EN translation tends to convert written numbers into digits (e.g., “trinta minutos” → “30 minutes”), while EN→PT does not. To preserve semantic symmetry, a digit-to-word mapping that rewrites numbers consistently in the canonical representation was introduced. Finally, we stored the original query, the detected language, and the normalized English form so downstream components always receive a stable and deterministic string for tokenization. Together, these fixes substantially reduced variation and improved the intent classification’s reliability.

D. Pre-loading of heavy components

Another crucial architectural decision in the pipeline is the pre-loading of the heavy components, namely the multilingual spaCy pipeline and the knowledge-graph, before any user query is processed. The spaCy model initialization involves loading the statistical components, taggers, dependency parsers, custom extension attributes and multilingual vocabulary vectors, as performing this at runtime for every query would be expensive. Similarly, the knowledge graph, sourced from a Turtle file exceeding one million lines, took too long to be reparsed from RDF on demand (33s for each query). To address this, the pipeline implements a PKL-based (ENCOMENDA REAL) caching system: on the first run, the TTL file is parsed into an in-memory graph structure, which is then serialized to disk as a .pkl snapshot. Subsequent executions detect the presence of the pickle file and load the graph directly and only once at the start in a fraction of the time. This strategy not only improves performance dramatically but also stabilizes memory usage, since the expensive graph structure is built exactly once and then reused.

The pipeline also introduces a dedicated mechanism that controls when entity extraction is executed, preventing unnecessary or harmful parsing for intents that do not require semantic grounding. For queries such as greetings, confirmations, or generic help requests, attempting entity extraction would lead to inconsistent results, noise in the entity linker, and contradictions between the detected intent and the returned slot set.

Conversation History of Recent Messages

To avoid exceeding the per-minute request limit and reduce computational costs, a compact history of the last three messages was implemented. While the full context of all the results in the entity extraction is given to groq for the current message, for the messages in the history only two fields are stored:

- *query* – the original user question,
- *response* – the final response returned by the LLM.

This history is used to generate a reduced context block appended at the end of the *prompt*. The messages appear in order of recency, with the most recent first. The number of messages included is configurable (default: 3). Placing the history at the end ensures that the trimming system always cuts from the tail, preserving the most relevant part of the main prompt.

As this implementation made the prompts longer, it was also necessary to add a trimming mechanism to the current query. This is done by beginning with the items found in the fuzzy matching with the lowest confidences. Then, for each of these items, the SPARQL results are eliminated one by one until the threshold of 7000 tokens is reached. This was the value decided because the model used for the experiments (chatgpt 120b) (ENCOMENDA REAL) has a per minute prompt limit of 8000.

E. Groq base prompt

Before each query is sent to Groq, a structured base prompt is provided that defines the model’s role, constraints, and formatting rules. This prompt positions the model as a bilingual PT/EN recipe assistant and explicitly instructs it to always answer in the same language as the user’s query—including translating tags, ingredients, steps, and recipe names when necessary. It also clarifies what contextual information the model is strictly allowed to use: the current query, up to three previous conversational messages, and any structured data extracted from the knowledge graph. Equally important, the prompt imposes strict behavioral guidelines: the model must rely solely on the provided structured data, avoid hallucinating missing information, and follow a modular answer format when multiple entities are detected (ingredients, tags, name). It also specifies how the model must handle meta-queries such as “what did I ask before?” by checking the last few conversational messages and replying strictly based on that context, if relevant. For off-topic queries, the prompt instructs

the model not to fabricate recipe results and instead provide a short clarification request or a single safe example query that can guide the user back to a valid intent. The final section of the prompt firstly enforces a conservative use of semantic description snippets (RAG), allowing them only when they are clearly relevant and never as a source for inventing missing structured data. Finally it also defines a discovery footer containing any secondary results that were not elaborated on in the main response. This prompt was built through iterative testing and refinement to ensure consistency and alignment with user expectations. After this base prompt, a structured block is appended containing:

- The pipeline result (intent, confidence, slots, SPARQL results);
- The top similarity snippets from the RAG index;
- The compact block of recent message history.

X. EXPERIMENTATION

A. Setup

We evaluated PT/EN queries by ingredients, tags, and time constraints.

B. Results

Positive cases showed accurate intent routing and bilingual output. Initial failures in time parsing and API configuration were mitigated through relaxed matching and environment validation.

Resultados

- Mostrar o quão boas as SPARQL queries foram a buscar os resultados desejados do dataset - resultado
- Refletir a introdução do RAG para a descrição no sistema, mesmo com as melhorias. - resultado
- Mostrar a interpretação das mensagens do historico + prompt + resultados do groq (receitas/ingredientes/tags/steps/time pedidos de acordo com a query) - uma tablea com exemplos na parte do prompt engineering

C. Metrics

- Intent accuracy (placeholder);
- Entity linking precision@k (placeholder);
- Time extraction success rate (placeholder);
- Human quality judgments for bilingual summaries (placeholder).

XI. CONCLUSIONS AND FUTURE WORK

We demonstrated that combining symbolic reasoning with neural generation produces accurate and fluent responses. The bilingual KG-powered assistant effectively supports multiple query types.

Future work includes:

- expanding KG coverage;
- adding automatic evaluation metrics;
- improving translation robustness;
- integrating structured outputs.

REFERENCES

- [1] G. Agrawal, T. Kumarage, Z. Alghamdi, and H. Liu, “Can knowledge graphs reduce hallucinations in llms?: A survey,” in *Proceedings of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024, accepted paper in NAACL 2024; also available as arXiv:2311.07914 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2311.07914>
- [2] “Course slides,” 2025, course materials, PowerPoint slides.
- [3] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, “spacy: Industrial-strength natural language processing in python,” <https://zenodo.org/record/1215370>, 2020, software release.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2019, submitted 11 Oct 2018 (v1); last revised 24 May 2019 (v2). [Online]. Available: <https://doi.org/10.48550/arXiv.1810.04805>
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *arXiv preprint arXiv:2005.11401*, 2020, accepted at NeurIPS 2020; v4. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [6] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, “Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” *arXiv preprint arXiv:1910.13461*, 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1910.13461>
- [7] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, eMNLP 2020; arXiv preprint. [Online]. Available: <https://doi.org/10.48550/arXiv.2004.04906>
- [8] T. Loesch, E. Durmuş, and R. Celebi, “Reciperag: A knowledge graph-driven approach to personalized recipe retrieval and generation,” in *Proceedings of RAGE-KG 2025*, 2025.
- [9] A. Bordes, N. Usunier, A. García-Durán, J. Weston, and O. Yakhnenko, “Translating embeddings for modeling multi-relational data,” in *Advances in Neural Information Processing Systems*, 2013, pp. 2787–2795.
- [10] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang, “Rotate: Knowledge graph embedding by relational rotation in complex space,” in *International Conference on Learning Representations (ICLR)*, 2019, accepted to ICLR 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1902.10197>
- [11] S. Zhang, Y. Tay, L. Yao, and Q. Liu, “Quaternion knowledge graph embeddings,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, accepted by NeurIPS 2019. [Online]. Available: <https://arxiv.org/abs/1904.10281>
- [12] C. Antelo, “Portuguese meals dataset,” <https://www.kaggle.com/datasets/catarinaantelo/portuguese-meals>, 2021, kaggle dataset; Accessed: 2025-12-04.
- [13] S. Li, “Food.com recipes and interactions dataset,” <https://www.kaggle.com/datasets/shuyangli94/food-com-recipes-and-user-interactions>, 2020, accessed: 2024-06-10.